Technical Assessment

AI Engineer

📋 Project Brief

Build a **Real-Time Analytics & Reporting Platform** that allows organizations to ingest data from multiple sources, visualize metrics through customizable dashboards, set up alerts, and generate scheduled reports. The application should demonstrate production-grade Python architecture, async processing, efficient data pipelines, and the ability to handle complex business logic at scale.

🏗️ Project Scope & Feature Details

The platform should function as a production-ready SaaS analytics tool (think a lightweight Mixpanel or Metabase). Below is a detailed breakdown of what each area should include:

🔐 Authentication & Multi-Tenancy

- Sign up / Sign in with email & password (passlib/bcrypt hashing)
- JWT access token (short-lived) + refresh token (HTTP-only cookie)
- OAuth2 integration (Google sign-in) — optional but recommended
- Organization creation during signup & invite-based team onboarding
- Role hierarchy: **Owner → Admin → Analyst → Viewer**
- Permission guards on API endpoints using dependency injection
- Organization-level data isolation at the database query layer

📊 Data Ingestion & Sources

- REST API endpoint for event ingestion (batch & single events)
- Support multiple data source types: API events, CSV uploads, webhook receivers
- Event schema validation using Pydantic models

- Async event processing via background task queue (Celery + Redis)
- Data normalization and storage in time-series optimized format
- Rate limiting on ingestion endpoints (per org, per API key)
- API key management — generate, revoke, rotate keys per organization

📈 Dashboards & Visualizations

- Create custom dashboards with drag-and-drop widget placement
- Widget types: line charts, bar charts, pie charts, KPI cards, tables
- Each widget connects to a saved query with configurable time range
- Dashboard sharing — public link (read-only) or team-only access
- Auto-refresh dashboards at configurable intervals (30s, 1m, 5m)
- Dashboard templates for common use cases (Web Analytics, Sales, DevOps)
- Full-screen presentation mode

🚨 Alerts & Notifications

- Define alert rules: metric threshold (e.g., "Error rate > 5% for 10 minutes")
- Alert evaluation via scheduled background tasks (Celery Beat)
- Notification channels: in-app, email, webhook (Slack-compatible)
- Alert history with timestamps and triggered values
- Mute/snooze alerts temporarily
- Alert status: Active, Triggered, Resolved, Muted

📋 Scheduled Reports

- Schedule recurring reports (daily, weekly, monthly) from any dashboard
- Generate PDF/PNG snapshots of dashboards
- Email reports to team members or external stakeholders
- Report generation via background tasks (Celery)
- Report history and download archive

⚡ Real-Time Features

- WebSocket-powered live dashboard updates (new events reflected immediately)
- Real-time alert notifications pushed to connected clients
- Live event stream viewer (tail incoming events in real-time)
- Connection state management and automatic reconnection

> ⚠️ **Scope Note**
>
> Candidates are **not expected to complete every feature listed**. Focus on delivering the "Must Have" modules with high code quality. The detailed scope above helps you understand the full vision so you can make informed architectural decisions that would support these features in a real production app.

🎯 Core Requirements

Module	Features	Priority
Authentication & Multi-Tenancy	JWT auth with refresh tokens, role-based access (Owner/Admin/Analyst/Viewer), org data isolation, invite system	Must Have
Data Ingestion	Event API (batch + single), CSV upload, Pydantic validation, async processing via Celery, API key management	Must Have
Dashboards & Widgets	Custom dashboards, chart widgets (line/bar/pie/KPI), configurable time ranges, sharing, auto-refresh	Must Have
Alerts & Notifications	Threshold-based alerts, Celery Beat scheduling, email + in-app + webhook notifications	Should Have
Real-Time Updates	WebSocket live dashboard refresh, real-time alert push, live event stream viewer	Should Have

🛠️ Technical Specifications

Frontend

- **Framework:** Next.js 14+ (App Router)
- **UI:** React 18+ with TypeScript
- **State:** Zustand or Redux Toolkit
- **Styling:** Tailwind CSS or Shadcn/UI

Backend (Python)

- **Framework:** FastAPI (preferred) or Django REST Framework
- **Language:** Python 3.11+ with type hints throughout

- **Charts:** Recharts, Chart.js, or D3.js
- **Data Fetching:** TanStack Query (React Query)
- **Real-Time:** WebSocket client (native or Socket.IO client)

- **Database:** PostgreSQL with SQLAlchemy 2.0 (async) or Django ORM
- **Migrations:** Alembic (FastAPI) or Django migrations
- **Task Queue:** Celery + Redis (with Celery Beat for scheduling)
- **Caching:** Redis for query results & rate limiting
- **Real-Time:** WebSockets via FastAPI/Starlette or Django Channels
- **Validation:** Pydantic v2 models
- **Testing:** pytest + pytest-asyncio + httpx (for async tests)

📐 Architecture Expectations

Design Pattern	Clean Architecture / layered separation (Routers → Services → Repositories → Models), dependency injection via FastAPI Depends
Async Architecture	Async/await throughout (async endpoints, async DB queries with SQLAlchemy 2.0), proper event loop handling
Error Handling	Custom exception classes, centralized exception handlers, structured error responses, proper logging (structlog)
Security	Input validation (Pydantic), CORS config, rate limiting (slowapi/redis), SQL injection prevention via ORM, XSS protection
Database Design	Proper indexing (esp. time-series queries), partitioning strategy for events table, migrations, seed scripts, soft deletes
Background Processing	Celery workers for heavy computation, Celery Beat for scheduled tasks, proper retry/backoff strategies, dead letter handling
Observability	Structured logging, request tracing (correlation IDs), health check endpoints, metrics exposure

📦 Deliverables

1. **GitHub Repository** — Clean commit history, monorepo or separate repos, proper branching
2. **README.md** — Setup instructions, architecture overview, environment variables
3. **Deployed Demo** — Live URL (Vercel for frontend, Railway/Render for backend + workers)

⭐ Evaluation Criteria

Criteria	Weight	What We Look For
Python Code Quality & Architecture	30%	Clean separation, type hints, async patterns, dependency injection, Pythonic idioms, SOLID principles
Functionality & Completeness	25%	All must-have features working, edge case handling, error states in UI, data pipeline reliability
UI/UX & Frontend	10%	Responsive design, chart interactions, loading states, optimistic updates, accessibility basics

💡 Bonus Points (Not Required)

- GraphQL API alongside REST (using Strawberry or Ariadne)
- OpenTelemetry instrumentation for distributed tracing
- Custom SQL query sandbox with query plan analysis
- Data retention policies with automatic archival
- Webhook delivery system with retry logic and delivery logs
- CI/CD pipeline (GitHub Actions) with lint, test, build, deploy stages
- Load testing results (Locust) demonstrating ingestion throughput
- Feature flags system for gradual rollouts

Questions about this assessment? Reach out to the hiring team. Good luck! 🚀